MRI ↔ CT CycleGAN

Complete Model Documentation

Full-scale bidirectional brain scan translation using unpaired CycleGAN.
Trained on Kaggle T4 GPU — 150 epochs — 256×256px — ~5000 images.

Parameter	Value
Architecture	CycleGAN (ResnetGenerator + NLayerDiscriminator)
Image Size	256 × 256 px
Training Epochs	150
Dataset	darren2020/ct-to-mri-cgan (Kaggle)
Total Images	~4974 (2486 CT + 2488 MRI)
Best Checkpoint	Epoch 110 (Val SSIM: 0.3771)
CT→MRI Test SSIM	0.264 ± 0.120
MRI→CT Test SSIM	0.486 ± 0.075

1. Project Overview

This project implements a CycleGAN to perform bidirectional image-to-image translation between MRI (Magnetic Resonance Imaging) and CT (Computed Tomography) brain scans. Both translation directions are trained simultaneously within a single model:

- CT → MRI: Convert CT scan appearance to MRI-style soft tissue contrast
- MRI → CT: Convert MRI scan appearance to CT-style bone/density contrast

CycleGAN was specifically chosen because the dataset is **unpaired** — CT and MRI images are not aligned by patient identity, scan session, or anatomical slice position. A supervised paired model such as pix2pix cannot be applied here because it requires corresponding image pairs at pixel level.

CycleGAN solves the unpaired problem by training two generators and two discriminators simultaneously, enforced by a cycle-consistency constraint: translating an image to the other domain and then back should reconstruct the original image faithfully. This provides a self-supervised training signal without needing paired data.

1.1 Project Pipeline

The project followed a structured four-phase methodology:

Phase	Description	Scale
Phase 1 — Encoder Search	Tested encoder hyperparameters, identified best 7 configs	Small
Phase 2 — Grid Search	28 full configurations × 10 epochs × 50 images	Small
Phase 3 — Confirmation	Top 4 configs × 30 epochs × 50 images	Small
Phase 4 — Full Training	Winning config × 150 epochs × full dataset × 256px	Full

1.2 Why CycleGAN Over Other Architectures

Several architectures were considered for this task:

Architecture	Paired?	Suitable Here?	Reason
pix2pix	Yes	No	Requires pixel-aligned pairs
CycleGAN	No	Yes	Works with unpaired domains
UNIT	No	Yes	Heavier, shared latent space
MUNIT	No	Yes	Multimodal, more complex
StarGAN	No	Partial	Better for 3+ domains
Diffusion Models	No	Possible	High compute, overkill for this scale

CycleGAN is the standard choice for unpaired medical image translation and has extensive literature validation on MRI/CT datasets specifically. The cycle-consistency loss provides strong geometric

preservation guarantees which are critical for medical imaging — a translated image must preserve anatomical structure.

2. Dataset

2.1 Source and Access

The dataset used is **darren2020/ct-to-mri-cgan** hosted on Kaggle. It contains brain scan images in both CT and MRI modalities, pre-split into train and test sets.

- **Kaggle path:** /kaggle/input/datasets/darren2020/ct-to-mri-cgan/Dataset/images/
- **Local path:** E:\code\mri to cti\Dataset\images\

2.2 Folder Structure

```
Dataset/images/
■■■ trainA/   – CT training images    (1742 images, ct*.png)
■■■ trainB/   – MRI training images   (1744 images, mri*.png)
■■■ testA/    – CT test images         (744 images)
■■■ testB/    – MRI test images        (744 images)
```

2.3 Data Split

The dataset already provides a train/test split via the trainA/trainB and testA/testB folders. An additional validation split was carved out at runtime from the training data:

Split	CT Images	MRI Images	Purpose
Train	1568	1570	Model weight updates each epoch
Validation	174	174	Val SSIM for best checkpoint selection
Test	744	744	Final evaluation only — never seen during training

> *Note: CT has 1742 and MRI has 1744 training images — a 2 image difference. The zip() in the training loop stops at the shorter domain (1568 CT batches per epoch), so 2 MRI images are skipped per epoch. Effect on training is negligible.*

2.4 Image Preprocessing

All images go through the following preprocessing pipeline before entering the network:

Step	Operation	Why
Resize	256×256 with BICUBIC interpolation	Standardize input size; BICUBIC preserves edge sharpness better tha
RandomHorizontalFlip	50% chance flip (experiment_v2 only)	Data augmentation; doubles effective dataset size
ToTensor	PIL [0,255] → float [0,1]	PyTorch tensor format
Normalize	mean=0.5, std=0.5 → maps [0,1] to [-1,1]	Matches generator Tanh output range

Images are loaded as RGB (3-channel) even though brain scans are grayscale. PIL's .convert('RGB') duplicates the single grayscale channel across R, G, B. This allows the same model architecture to be used regardless of source modality.

3. Hyperparameter Search

3.1 Phase 2 — 28-Run Grid Search

After identifying the best 7 encoder configurations in Phase 1, a full grid search was run combining encoder parameters with decoder/discriminator parameters. Each of the 28 runs was trained for 10 epochs on 50 images per domain at 128px resolution.

Each run produced a train_log.txt and config.json. A summary CSV was generated by computing statistics over the **last 20 log entries** of each run (not all entries — confirmed by cross-referencing CSV values against log files).

Scoring metric:

```
combined_score = lossG_mean_last20 + lossD_mean_last20
               + G_stability + D_stability
```

Lower combined_score = better. This rewards low average loss AND stable training (low variance between log entries).

3.2 Phase 3 — 30-Epoch Confirmation

Top 4 configs from Phase 2 were run for 30 epochs on 50 images. Visual inspection of generated images at epoch 30 was the primary selection criterion:

Config	Rank	Outcome at Epoch 30
confirm_run_012	1st	Winner — clean images, stable losses, no artifacts
confirm_run_018	2nd	Grid artifacts on fakeB (n_blocks=6 caused issues)
confirm_run_002	3rd	Severe checkerboard + noise artifacts
confirm_run_004	5th	Severe checkerboard + noise artifacts

3.3 Winning Configuration — confirm_run_012

Hyperparameter	Value	Description
lr_G	0.0001	Generator learning rate
lr_D	0.0001	Discriminator learning rate
lambda_cycle	5.0	Cycle-consistency loss weight
lambda_identity	2.5	Identity loss weight
n_blocks	4	Residual blocks in generator
n_layers_D	3	PatchGAN discriminator depth
use_spect	True	Spectral normalization on discriminator
ngf / ndf	64	Generator / discriminator base feature channels

Hyperparameter	Value	Description
batch_size	1	Images per training step
image_size	128 → 256	Search at 128px, full training at 256px

4. Model Architecture

4.1 CycleGAN Overview — Four Networks

CycleGAN trains four networks simultaneously:

Network	Role	Input	Output
G_A2B	Generator CT→MRI	CT image (3×256×256)	Fake MRI (3×256×256)
G_B2A	Generator MRI→CT	MRI image (3×256×256)	Fake CT (3×256×256)
D_A	Discriminator CT	CT or Fake CT (3×256×256)	Patch map (1×30×30)
D_B	Discriminator MRI	MRI or Fake MRI (3×256×256)	Patch map (1×30×30)

4.2 Generator — ResnetGenerator

The generator follows the Encoder → Residual Blocks → Decoder architecture from the original CycleGAN paper (Johnson et al. 2016). It processes the full image at multiple scales and must output a complete 256×256 image.

Architecture Layers

```
Input: 3 × 256 × 256

[Encoder]
ReflectionPad2d(3)
Conv2d(3 → 64,  kernel=7, stride=1)  + InstanceNorm + ReLU
Conv2d(64 → 128, kernel=3, stride=2)  + InstanceNorm + ReLU
Conv2d(128 → 256, kernel=3, stride=2) + InstanceNorm + ReLU

[Bottleneck — 4 Residual Blocks @ 256 channels]
ResBlock × 4

[Decoder]
ConvTranspose2d(256 → 128, stride=2, output_padding=1) + InstanceNorm + ReLU
ConvTranspose2d(128 → 64,  stride=2, output_padding=1) + InstanceNorm + ReLU
ReflectionPad2d(3)
Conv2d(64 → 3, kernel=7) + Tanh

Output: 3 × 256 × 256  (range [-1, 1])
```

Residual Block Detail

```
def forward(x):
    residual = x
    x = ReflectionPad2d(1) → Conv2d(256,256,3) → InstanceNorm → ReLU
    x = ReflectionPad2d(1) → Conv2d(256,256,3) → InstanceNorm
    return x + residual   # skip connection
```

Design Choices Explained

Choice	Alternative	Why This Choice
ReflectionPad2d	Zero padding	Avoids border artifacts — mirrors edge pixels instead of inserting zeros
InstanceNorm	BatchNorm	BatchNorm with batch_size=1 is equivalent to no normalization. InstanceNor
ConvTranspose2d	Bilinear upsample + Conv	Learnable upsampling; combined with spectral norm on discriminator preven
Tanh output	Sigmoid	Output range [-1,1] matches normalized input range — consistent with the [-1
n_blocks=4	9 (paper default)	Chosen from hyperparameter search at 128px. Paper recommends 9 for 256

Total parameters: 5,477,379

4.3 Discriminator — NLayerDiscriminator (PatchGAN)

The discriminator does not classify the full image as real or fake. Instead it outputs a spatial patch map where each value represents the realness score of a local image region. With n_layers_D=3 at 256px input, the output is 30×30 — each patch value covers approximately 70×70 pixels of the input.

This patch-based approach has several advantages over a full-image discriminator: it operates on local texture and structure rather than global composition, is faster to compute, and produces sharper high-frequency details in generated images.

Architecture Layers

```
Input: 3 × 256 × 256

Conv2d(3 → 64,   stride=2) + SpectralNorm + LeakyReLU(0.2)    [no InstanceNorm on first
 layer]
Conv2d(64 → 128,  stride=2) + SpectralNorm + InstanceNorm + LeakyReLU(0.2)
Conv2d(128 → 256, stride=2) + SpectralNorm + InstanceNorm + LeakyReLU(0.2)
Conv2d(256 → 512, stride=1) + SpectralNorm + InstanceNorm + LeakyReLU(0.2)
Conv2d(512 → 1,   kernel=4, padding=1) + SpectralNorm

Output: 1 × 30 × 30  (patch realness scores)
```

Spectral Normalization

use_spect=True applies spectral normalization to every convolutional layer in the discriminator (including the final layer). Spectral norm constrains the Lipschitz constant of the discriminator by normalizing weights by their largest singular value. This:

- Prevents the discriminator from becoming too powerful relative to the generator
- Stabilizes GAN training — less oscillation in loss values
- Was a key finding from the hyperparameter search: all top configs had use_spect=True

Total parameters: 2,764,737

Why Generator Has More Parameters Than Discriminator

The generator must reconstruct a complete 256×256 image pixel by pixel — it has an encoder, 4 residual blocks, and a full decoder. The discriminator only needs to output a 30×30 grid of scores — a much simpler task requiring far less capacity. The asymmetry in task complexity directly causes the 5.4M vs 2.7M parameter difference.

4.4 Weight Initialization

All convolutional layers are initialized with a normal distribution (mean=0.0, std=0.02). InstanceNorm layers are initialized with normal(mean=1.0, std=0.02) for weights and 0 for bias. This is the standard CycleGAN initialization. Small random weights at std=0.02 (smaller than the PyTorch default of 0.01 * kaiming) help prevent symmetry-breaking issues where all neurons learn the same features at the start of training.

5. Training Configuration

5.1 Loss Functions

Adversarial Loss — LSGAN (Least Squares GAN)

Standard GAN loss uses Binary Cross-Entropy (BCE). LSGAN replaces it with Mean Squared Error (MSE):

```
# Generator wants discriminator to output 1 (thinks it is real)
loss_GAN = MSE(D(G(x)), ones_like(D(G(x))))

# Discriminator: real images → 1, fake images → 0
loss_D = 0.5 * (MSE(D(real), 1) + MSE(D(fake), 0))
```

LSGAN is more stable than BCE-GAN because it penalizes samples that are on the correct side of the decision boundary but far from it — BCE saturates and produces near-zero gradients for confident predictions, while MSE maintains gradient signal throughout training.

Cycle-Consistency Loss

The core constraint that enables unpaired training:

```
loss_cycle = L1(G_B2A(G_A2B(real_CT)),  real_CT)  * lambda_cycle
           + L1(G_A2B(G_B2A(real_MRI)), real_MRI) * lambda_cycle

lambda_cycle = 5.0
```

Translating real_CT → fake_MRI → reconstructed_CT should recover the original CT image. This forces the generators to learn a meaningful mapping rather than ignoring the input. lambda_cycle=5.0 was the winning value from the search — strong enough to enforce geometry without overwhelming the adversarial signal.

Identity Loss

```
loss_identity = L1(G_B2A(real_CT),  real_CT)  * lambda_identity
              + L1(G_A2B(real_MRI), real_MRI) * lambda_identity

lambda_identity = 2.5  (= lambda_cycle / 2, standard ratio)
```

When a generator receives an image already in its target domain, it should return that image unchanged. This preserves overall color and structure consistency. The standard ratio is lambda_identity = 0.5 * lambda_cycle.

> *Note: For CT↔MRI translation, the two modalities look very different. Identity loss may be too constraining for CT→MRI, preventing the generator from making the large textural changes needed. This is being investigated in experiment_v2 with lambda_identity=0.5.*

Total Generator Loss

```
loss_G = loss_GAN_A2B + loss_GAN_B2A
       + loss_cycle_A + loss_cycle_B
       + loss_idt_A   + loss_idt_B
```

5.2 Optimizers

Both generators share one Adam optimizer; both discriminators share one Adam optimizer. One optimizer.step() updates both generators simultaneously, one updates both discriminators.

Optimizer	Networks	lr	beta1	beta2
optimizer_G	G_A2B + G_B2A	0.0001	0.5	0.999
optimizer_D	D_A + D_B	0.0001	0.5	0.999

beta1=0.5 (vs PyTorch default 0.9) is standard for GAN training. Lower beta1 means the optimizer forgets momentum faster, which helps with the non-stationary loss landscape of adversarial training where the target keeps moving.

5.3 Learning Rate Schedule — Linear Decay

Constant LR for first half, linear decay to zero for second half:

```
def lr_lambda(epoch):
    e = epoch + 1  # LambdaLR is 0-indexed internally
    if e < decay_epoch:    # epochs 1-75: constant
        return 1.0
    # epochs 75-150: linear decay to 0
    return max(0.0, 1.0 - (e - decay_epoch) / (total_epochs - decay_epoch))
```

The LR at epoch 80 (where session 2 crashed) was approximately 0.0000933. The LR reaches exactly 0.000000 at epoch 150 (confirmed in training logs).

5.4 Image Replay Buffer

The replay buffer stores the last 50 generated images per domain. During discriminator training, each fake image has a 50% probability of being replaced by a randomly selected stored image from the buffer:

```
def push_and_pop(images):
    for img in images:
        if buffer not full:
            add to buffer, return current
        else if random() > 0.5:
            swap random stored image with current
            return stored image  # discriminator sees older fake
        else:
            return current fake
```

This exposes the discriminator to a diverse history of generator outputs rather than only the most recent batch, preventing the discriminator from overfitting to the current generator state and stabilizing the adversarial dynamic.

6. Code Walkthrough — kaggle_notebook.py

The full training is implemented as a single Python script organized into 12 sections. Each section is described below with its purpose, key decisions, and any non-obvious behavior.

Section 1 — Setup and Imports

Imports all required libraries and sets random seeds across Python, NumPy, and PyTorch for full reproducibility. SEED=42 is applied to all random number generators. Device detection runs at startup — if CUDA is unavailable the script falls back to CPU but will be extremely slow at 256px.

- torch.cuda.manual_seed_all(SEED) — seeds all CUDA devices, not just the current one
- skimage.metrics: structural_similarity and peak_signal_noise_ratio used for evaluation only

Section 2 — Dataset Path Explorer

Walks the Kaggle input directory up to 4 levels deep and prints the folder tree. This section exists purely for verification before any training code runs. Running this cell first is how the correct dataset paths were discovered — the Kaggle input path includes the dataset owner username (darren2020) which is not immediately obvious from the dataset page URL.

Section 3 — Configuration

All hyperparameters and file paths live in a single CONFIG dictionary. Nothing is hardcoded elsewhere in the script — all downstream sections read from CONFIG. This makes the notebook reproducible: changing one CONFIG entry propagates everywhere.

Three output directories are created at the start of this section:

- /kaggle/working/outputs/ — root output directory
- /kaggle/working/outputs/checkpoints/ — model weight files (.pth)
- /kaggle/working/outputs/samples/ — sample grid images per epoch

Section 4 — Dataset

MRICTDataset class

Loads all image files from a flat directory (no subdirectories expected). Files are sorted alphabetically for deterministic ordering across runs. Images are converted to RGB on load using PIL's .convert('RGB') — this handles grayscale source files by duplicating the single channel across R, G, B, making the dataset compatible with the 3-channel model architecture.

Raises RuntimeError immediately on empty directory rather than silently running on zero data — a fast-fail approach that surfaces path issues early.

split_dataset function

PyTorch's random_split is seeded with a fixed Generator to ensure the same 90/10 train/validation split every time the notebook runs. This is critical for multi-session training — if the split were random, different sessions would have different validation sets, making SSIM comparisons meaningless.

DataLoaders

Training loaders use shuffle=True (random order each epoch, good for SGD) and pin_memory=True (locks tensor memory pages for faster CPU→GPU transfer). Validation and test loaders use shuffle=False — consistent ordering is required so that zip(loader_src, loader_tgt) matches the same positional index each time for SSIM computation.

Section 5 — Model Architecture

Defines all four network classes and the init_weights function. All models are instantiated, moved to device (.to(device)), and weight-initialized in this section. Parameter counts are printed to confirm the architecture is correct before training begins.

Section 6 — Losses, Optimizers, Schedulers

gan_loss function

Creates a target tensor of ones (real) or zeros (fake) matching the discriminator output shape using torch.ones_like / torch.zeros_like. This automatically handles whatever spatial dimensions the discriminator produces without hardcoding shape. MSE between prediction and target implements LSGAN loss.

lr_lambda and the 0-index offset

PyTorch's LambdaLR scheduler internally tracks an epoch counter starting at 0 (not 1). The lambda function receives this 0-indexed value. Since our training loop uses 1-indexed epochs (range(1, 151)), comparisons like 'epoch < decay_epoch' would be off by one without correction. The fix: e = epoch + 1 at the start of lr_lambda maps the 0-indexed internal counter to 1-indexed training epochs correctly.

Section 7 — Image Replay Buffer

Implements the history buffer from the original GAN training paper (Shrivastava et al., 2017). The buffer uses a plain Python list rather than collections.deque because the random swap operation requires integer index access (buffer[idx] = img) — deque supports this syntax but at O(n) cost and with unexpected behavior for assignment. See Error 3 in Section 8.

Section 8 — Validation SSIM

compute_val_ssim iterates over validation loader pairs using zip() for positional matching. Critical details:

- G.eval() is called before inference to disable training-mode behaviors (dropout if present, BatchNorm running stats). G.train() restores training mode after.
- Images are denormalized from [-1,1] to [0,1] before SSIM computation: (tensor + 1) / 2

- channel_axis=2 in skimage's ssim because numpy arrays after transpose are H×W×C
- data_range=1.0 because images are in [0,1] range after denormalization

Section 9 — Training Loop

Per-batch training order

Within each batch, generators are trained first, then discriminators:

```
1. optimizer_G.zero_grad()
   compute identity losses: G_B2A(real_CT)≈real_CT, G_A2B(real_MRI)≈real_MRI
   compute GAN losses:      D_B(G_A2B(real_CT))→1, D_A(G_B2A(real_MRI))→1
   compute cycle losses:    G_B2A(fake_MRI)≈real_CT, G_A2B(fake_CT)≈real_MRI
   total loss_G.backward()
   optimizer_G.step()

2. optimizer_D.zero_grad()
   D_A: MSE(D_A(real_CT),1) + MSE(D_A(buffer_fake_CT),0)
   D_B: MSE(D_B(real_MRI),1) + MSE(D_B(buffer_fake_MRI),0)
   loss_D.backward()
   optimizer_D.step()
```

Important: fake_A and fake_B computed during generator training are reused for cycle loss — they are not recomputed. This is efficient and correct. The discriminator receives these tensors via .detach() which removes them from the computation graph, preventing gradients from flowing back through the generator during discriminator training.

Resume logic cascade

On startup, the training loop checks three locations in priority order:

```
1. save_resume_path  = /kaggle/working/outputs/checkpoints/resume_state.pth
   (current session's checkpoint — highest priority, most recent)

2. full_resume_path  = /kaggle/input/datasets/.../resume_state.pth
   (uploaded from previous session — full state: weights + optimizer + scheduler)

3. partial_resume_dir = /kaggle/input/.../cycle-gan-resume-checkpoint-80/
   (individual weight files when resume_state.pth was unavailable)
```

Partial resume (path 3) loads model weights only — optimizer states are restarted fresh and the scheduler is fast-forwarded by calling step() 79 times before training begins. History is pre-filled with NaN for the missing epochs to keep the x-axis correct in plots.

Section 10 — Loss Curves

Plots three subplots: (1) generator and discriminator loss over all training epochs, (2) validation SSIM for both directions over validation epochs, (3) CT→MRI SSIM with a vertical line marking the LR decay start epoch. val_epochs is derived from actual history length rather than hardcoded, to handle partial sessions where history has fewer entries than expected.

Section 11 — Test Set Evaluation

Loads the best checkpoint weights (G_A2B_best.pth and G_B2A_best.pth), runs inference over the full 744-image test set, and computes MAE, SSIM, and PSNR per image. Reports mean ± standard deviation for each metric. Results are saved to test_metrics.json for reproducible reference. map_location=device is specified on torch.load to handle cases where the checkpoint was saved on a GPU but the evaluation runs on a different device.

Section 12 — Visual Results

show_results generates a 3-row × 6-column figure: row 1 = input images, row 2 = generated images, row 3 = real target images. Run for both CT→MRI and MRI→CT directions on the test set. Saved as PNG files in the outputs directory. Provides a qualitative sanity check complementing the quantitative metrics.

7. Training Process and Sessions

Full training required 3 sessions across multiple days due to Kaggle's 9-hour session limit. Each session is documented below including issues encountered.

7.1 Session 1 — Epochs 1 to 70

Item	Detail
Kaggle Version	Version 1
GPU	T4 (16GB VRAM)
Duration	~9 hours (session timeout)
Epochs Completed	1–70 (72 seen in logs but last checkpoint at 70)
Outcome	Clean run — natural timeout
Last Checkpoint	resume_state.pth at epoch 70
Best SSIM	0.3577 at epoch unknown within session

Epoch timing: epoch 1 completed at 620 seconds (~10.3 minutes). All subsequent epochs consistent at ~10 min/epoch. This gave the estimate of ~35-40 epochs per 9-hour session, ~4 sessions total.

7.2 Session 2 — Epochs 71 to 80 (Crashed)

Item	Detail
Kaggle Version	Version 2
GPU	T4
Resumed From	Epoch 70 via uploaded resume_state.pth
Crash Point	Epoch 80 — during resume_state.pth save
Error	RuntimeError: open file failed — Read-only file system
Files Saved Before Crash	G_A2B_epoch80.pth, G_B2A_epoch80.pth, G_A2B_best.pth, G_B2A_best.pth, D_A_best.pth, D_B_
Best SSIM	0.3602 at epoch 80

The crash occurred because the resume path variable was overwritten with the read-only Kaggle input path when falling back to the uploaded checkpoint. See Error 2 in Section 8 for full details and fix.

7.3 Session 3 — Aborted (No GPU)

Item	Detail
Kaggle Version	Version 3
GPU	None — CPU only

Item	Detail
Duration	~25 minutes (aborted manually)
Outcome	Detected Device: cpu in logs, stopped immediately
Cause	GPU accelerator not re-enabled when creating new session

7.4 Session 3 (Retry) — Epochs 81 to 150

Item	Detail
Kaggle Version	Version 4
GPU	T4
Resume Type	Partial — individual weight files from epoch 80
Files Loaded	G_A2B_best.pth, G_B2A_best.pth, D_A_best.pth, D_B_best.pth
Optimizer State	Restarted fresh — Adam momentum lost
Scheduler	Fast-forwarded 79 steps to match epoch 80 LR position
LR at Start	0.000093 (correct for epoch 81)
Duration	~11 hours
Outcome	Completed successfully — epoch 150 reached
Best SSIM	0.3771 at epoch 110

7.5 Session Timeline Summary

Session	Version	Epochs	Outcome
1	V1	1 → 70	Clean run — natural timeout at 9 hours
2	V2	71 → 80	Crashed saving checkpoint — read-only path bug
3 attempt 1	V3	—	Aborted — CPU only, no GPU
3 attempt 2	V4	81 → 150	Completed — training finished at epoch 150

8. Errors and Fixes

Eight bugs or mistakes were identified and fixed during the course of development and training. Each is documented with root cause analysis and the exact fix applied.

Error 1 — Wrong Dataset Path

What went wrong:

The initial dataset path assumed /kaggle/input/ct-to-mri-cgan/ with subdirectories train/CT and train/MRI. The actual Kaggle input path was: /kaggle/input/datasets/darren2020/ct-to-mri-cgan/Dataset/images/trainA

Why it happened: Kaggle dataset input paths include the dataset owner's username (darren2020) which does not appear in the dataset URL. The internal folder structure inside the dataset (Dataset/images/trainA vs expected dataset/image/train/CT) was also non-standard and not visible without running the notebook.

Additionally, the folder tree depth was limited to 2 levels (if depth > 2: continue), which hid trainA/trainB/testA/testB folders that exist at depth 3. The tree showed 'images/' folder but appeared to have 0 files inside it.

Fix:

User ran the Kaggle default starter cell (os.walk('/kaggle/input')) which prints all file paths. Actual paths were read from the output and updated in CONFIG. Folder tree depth limit was changed from > 2 to > 4 to show all subfolders.

Error 2 — resume_state.pth Saved to Read-Only Input Path (Session 2 Crash)

What went wrong:

Session 2 crashed at epoch 80 with:

```
RuntimeError: [enforce fail at inline_container.cc:743]
open file failed with strerror: Read-only file system
```

The code tried to write resume_state.pth to /kaggle/input/... which is read-only.

Why it happened: A single variable resume_path was used for both loading and saving the checkpoint. When the code fell back to the uploaded checkpoint (at /kaggle/input/...), it overwrote resume_path with that path. Later, torch.save(..., resume_path) tried to write to the same location — which is in the read-only input filesystem.

Fix:

Split into two separate variables:

```
save_resume_path = '/kaggle/working/outputs/checkpoints/resume_state.pth'
load_resume_path = None  # determined at runtime

# Load from working dir first, then input dir
if exists(save_resume_path):    load_resume_path = save_resume_path
```

```
elif exists(full_resume_path):  load_resume_path = full_resume_path

# Save always goes to working dir
torch.save(state, save_resume_path)
```

Error 3 — deque Used With Integer Indexing in ReplayBuffer

What went wrong:

The ReplayBuffer was implemented using collections.deque with maxlen=50, then accessed with integer indexing: tmp = self.buffer[idx].clone() and self.buffer[idx] = img. Python's deque supports [] access but it is O(n) and assignment to a deque by index produces incorrect behavior when the buffer is full (the maxlen auto-drop behavior interacts badly with index-based mutation).

Why it happened: deque was chosen for its maxlen feature which automatically drops old elements. But the random-swap logic requires direct integer index access and assignment, which is the intended use case for a list, not a deque.

Fix:

Replaced deque with a plain Python list. Manual size management: only append when len(buffer) < max_size, randomly replace an existing element when full. The deque import was also removed as it became unused.

Error 4 — val_epochs Length Mismatch in Matplotlib Plots

What went wrong:

Section 10 used a hardcoded x-axis: val_epochs = list(range(10, CONFIG['epochs']+1, 10)) — always 15 entries for 150 epochs. After partial resume from epoch 80, history['val_ssim_A2B'] only had entries for epochs 90, 100, 110, 120, 130, 140, 150 — 7 entries. Plotting 15 x-values against 7 y-values would crash matplotlib with a length mismatch.

Why it happened: The x-axis was hardcoded based on total training epochs rather than derived from actual history. This worked for the first complete session but broke for partial sessions where history starts mid-training.

Fix:

```
# Before:
val_epochs = list(range(10, CONFIG['epochs'] + 1, 10))

# After:
val_epochs = list(range(10, 10 * len(history['val_ssim_A2B']) + 1, 10))
```

Error 5 — LR Scheduler Warning in Partial Resume Advancement Loop

What went wrong:

PyTorch 1.1+ warns that scheduler.step() must be called after optimizer.step(). The advancement loop called scheduler.step() 79 times before any optimizer.step() had occurred, generating repeated warning

messages in the logs.

Why it happened: The advancement loop is not doing real training — it is only fast-forwarding the scheduler's internal epoch counter to the correct position for resuming. No optimizer step exists or is needed. PyTorch's warning is technically correct for normal training but does not apply here.

Fix:

```
import warnings
with warnings.catch_warnings():
    warnings.simplefilter('ignore')
    for _ in range(partial_resume_epoch - 1):
        scheduler_G.step()
        scheduler_D.step()
```

Error 6 — Scheduler Advanced One Extra Step (Off-by-One)

What went wrong:

The partial resume code advanced the scheduler 80 times before training epoch 81. The training loop also calls scheduler.step() at the end of every epoch. So after epoch 81 completed, the scheduler had been stepped 81 times total — one epoch ahead of the actual training epoch. This persisted throughout session 3, meaning the LR decay curve was shifted one epoch earlier than intended.

Why it happened: The advancement count matched partial_resume_epoch (80) without accounting for the training loop's own scheduler.step() call at epoch 81 end. The total after epoch 81 = 80 pre-advance + 1 loop step = 81, but epoch 81 position should correspond to 80 total steps.

Fix:

```
# Before:
for _ in range(partial_resume_epoch):      # 80 steps

# After:
for _ in range(partial_resume_epoch - 1):  # 79 steps
# Loop adds step 80 at end of epoch 81 — total = 80 (correct)
```

Error 7 — Session Started Without GPU

What went wrong:

Version 3 ran with Device: cpu instead of Device: cuda. Training at 256px on CPU would take approximately 100+ minutes per epoch versus ~10 minutes on T4 GPU — making 70 remaining epochs take ~120 hours on CPU.

Why it happened: When creating a new Kaggle version (Save & Run All), the GPU accelerator setting is not automatically preserved from the previous version. It must be manually re-enabled each time under Settings → Accelerator → GPU T4.

Fix:

The session was stopped within 25 minutes after seeing Device: cpu in logs. T4 GPU re-enabled in Kaggle Settings, new version created (Version 4) and rerun.

Note: Always verify 'Device: cuda' in the first cell output before closing the laptop. If Device: cpu appears, stop immediately and check accelerator settings.

Error 8 — lr_lambda Off-by-One Epoch Indexing

What went wrong:

PyTorch's LambdaLR scheduler internally tracks an epoch counter starting at 0 (not 1 — it increments before calling the lambda on the first step). The original lr_lambda compared this 0-indexed value directly to decay_epoch=75 (which is 1-indexed). The result: LR decay began at internal epoch 75 which corresponds to training epoch 76 — one epoch late.

Why it happened: Mismatch between PyTorch's internal 0-indexed epoch counter and the 1-indexed epoch variable (range(1, 151)) used in the training loop. Easy to miss because the effect is subtle — LR decay starts one epoch late rather than crashing.

Fix:

```
def lr_lambda(epoch):
    e = epoch + 1  # convert 0-indexed to 1-indexed
    if e < CONFIG['decay_epoch']:
        return 1.0
    return max(0.0, 1.0 - (e - CONFIG['decay_epoch']) /
               (CONFIG['epochs'] - CONFIG['decay_epoch']))
```

9. Results

9.1 Test Set Metrics

Evaluation was performed on the held-out test set (744 CT + 744 MRI images) using the best checkpoint saved at epoch 110 (Val SSIM: 0.3771).

Direction	MAE (↓ better)	SSIM (↑ better)	PSNR dB (↑ better)
CT → MRI	0.169 ± 0.070	0.264 ± 0.120	12.40 ± 2.80
MRI → CT	0.173 ± 0.040	0.486 ± 0.075	9.78 ± 1.20

9.2 Training Dynamics

Metric	Epoch 81 (session 3 start)	Epoch 110 (best)	Epoch 150 (final)
Loss_G	2.235	2.177	1.904
Loss_D	0.112	0.104	0.131
Val SSIM A2B	—	0.274 (best)	0.267
Val SSIM B2A	—	0.480 (best)	0.475
LR	0.000093	0.000053	0.000000

Key observations from training dynamics:

- Generator loss declined steadily from 2.71 (epoch 1) to 1.90 (epoch 150) — healthy monotonic improvement throughout training
- Discriminator loss dropped from 0.44 to ~0.07 by epoch 100, then rose back to 0.13 by epoch 150 — this rise is positive, indicating the generator became strong enough to challenge the discriminator again
- Val SSIM peaked at epoch 110 for both directions — best checkpoint correctly captured
- MRI→CT SSIM (0.45-0.48) consistently higher than CT→MRI (0.25-0.27) throughout all epochs
- LR decay visible in loss curves: smoother, steadily declining from epoch 130 onward

9.3 Visual Quality Analysis

Sample images were saved every 10 epochs during session 3 (epochs 90-150). Grid layout per image: [real CT | fake MRI | real MRI | fake CT]

MRI → CT Direction (columns 3-4)

- Clearly better quality — skull boundary well defined from epoch 90
- Bone density contrast visible and improving epoch by epoch
- By epoch 150: structurally very close to real CT scans
- Low std of SSIM (0.075) confirms consistent quality across test images

CT → MRI Direction (columns 1-2)

- Brain structures visible but soft tissue contrast is flat and dark
- MRI's bright white matter / dark grey matter distinction not fully captured
- Slight left-side geometric asymmetry present from epoch 70, persists to epoch 150
- High std of SSIM (0.120) indicates inconsistent quality across test images

9.4 Why CT→MRI is Harder

The performance gap between directions is expected and has a clear cause:

- CT encodes tissue density (Hounsfield units) — bone is bright, soft tissue is grey, air is black. The mapping from CT to MRI requires the generator to hallucinate soft tissue texture that is not encoded in the CT signal
- MRI encodes proton density and relaxation times — the mapping to CT bone structure is more direct because bone has predictable density in CT
- MRI scan type variability (T1, T2, FLAIR) in the training data may cause the generator to average across styles rather than produce a specific MRI type
- n_blocks=4 may be insufficient for the complex CT→MRI texture generation at 256px

10. Assessment and Next Steps

10.1 Current Model Limitations

Limitation	Evidence	Planned Fix
CT→MRI SSIM too low (0.264)	Visible in both metrics and visual inspection	Increase n_blocks from 4 to 9
Geometric asymmetry in generated images	Left side of brain slightly deformed in epoch 70	Increase lambda_cycle from 5.0 to 10.0
n_blocks=4 selected at 128px/50 images	Hyperparameter search not representative of 256px dataset	Retrain at 256px and directly test n_blocks=9
Identity loss may over-constrain CT→MRI	CT and MRI look very different — identity loss forces unnecessary similarity	Test lambda_identity=0.5 in experiment_v2
Training stopped at epoch 150	Loss still declining at epoch 150 (1.904), still dropping	Train for 200 epochs in next run

10.2 Experiment V2 — Local Testing

Before committing to a full 200-epoch Kaggle run, three configurations are being tested locally on 500 images × 30 epochs to validate improvements cheaply (approximately 1 hour per config on RTX 3050 6GB Laptop GPU):

Config	n_blocks	lambda_cycle	lambda_identity	Flip Aug	Purpose
A (baseline)	4	5.0	2.5	No	Reference — current model config
B	9	10.0	2.5	Yes	Architecture + cycle + augmentation
C	9	10.0	0.5	Yes	Same as B but reduced identity loss

If Config B or C achieves higher Val SSIM than Config A at epoch 30, the winning config will be used for a full 200-epoch Kaggle training run.

Script location: E:\code\mri to cti\experiment_v2\train_experiment.py

10.3 File Reference

File / Folder	Description
E:\code\mri to cti\kaggle_notebook.py	Full Kaggle training notebook (Sections 1-12)
E:\code\mri to cti\experiment_v2\train_experiment.py	Local experiment script — 3 configs × 30 epochs
E:\code\mri to cti\results after epoch 150	Complete output from final Kaggle training run
E:\code\mri to cti\backup\full parameter test	28-run grid search results (10 epochs each)
E:\code\mri to cti\backup\epooch 30 test	30-epoch confirmation run results (top 4 configs)
E:\code\mri to cti\resume_state.pth	Checkpoint file from session 1 end (epoch 70)

10.4 Comparison Baseline

For academic comparison, this model's scores can be benchmarked against:

- Other notebooks on the same Kaggle dataset: darren2020/ct-to-mri-cgan → Notebooks tab
- Fair comparison requires: same testA/testB split, same 256px size, same SSIM definition
- Published CycleGAN papers on MRI/CT typically report SSIM of 0.30-0.55 depending on direction — MRI→CT at 0.486 is competitive; CT→MRI at 0.264 is below average
- Pix2pix notebooks on the same dataset will show higher SSIM (0.6-0.8) because they use paired supervision — not a fair comparison to unpaired CycleGAN

End of Documentation